STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0217	PIKA

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity 2. Pumps – 3 Quantity 3. FT-01 Flow Transmitter – 01 Quantity 4. Ball valve – 3 Quantity 5. ON/OFF/XV Valve – 1 Quantity 6. 10-mm OD tubing/blue – 9 meters/1 meter 7. Microcontroller/PCB/ data-acquisition software 8. Required electrical/ electronic accessories	1. Custom PCB 2. ESP32 NodeMCU module 3. Pump Driver-1 4. Pump Driver-2 5. Pump Driver-3 6. 12 V DC power input (**SMPS**) 7. 12 VDC Solenoid-valve interface 8. Flow-sensor interface 9. Float-switch interface 10. ON/OFF-switch interface 11. Pump-driver PWM/enable interfaces 12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code.** The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

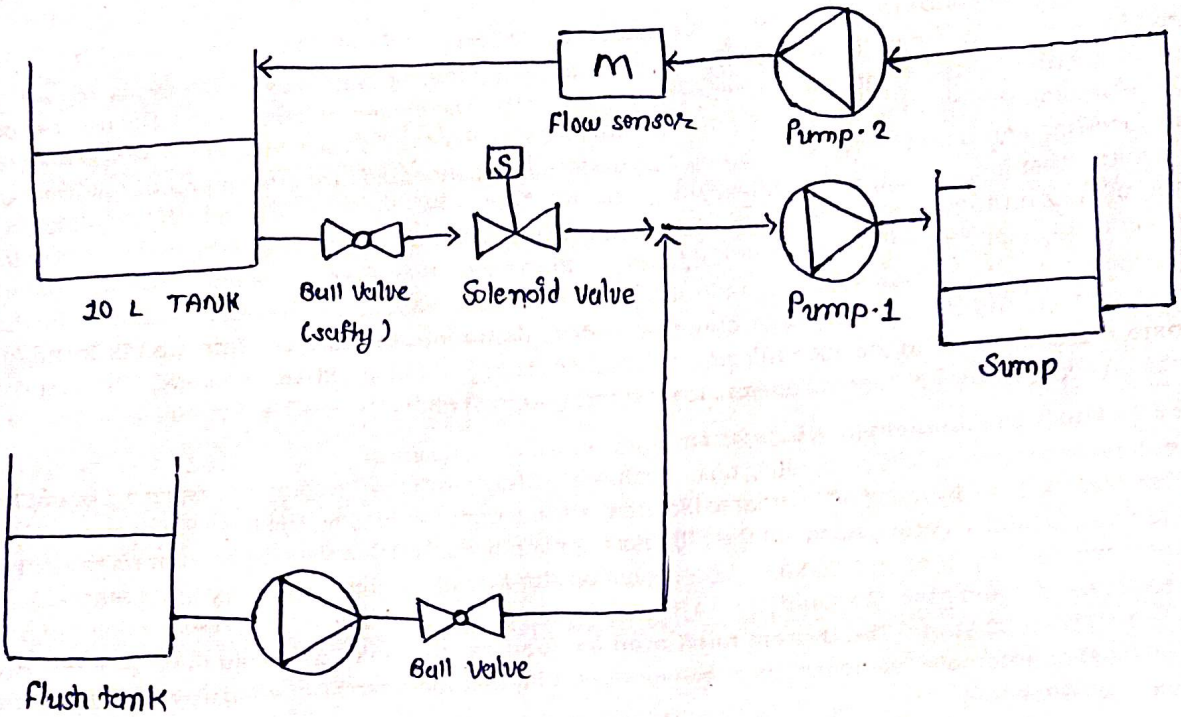
A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

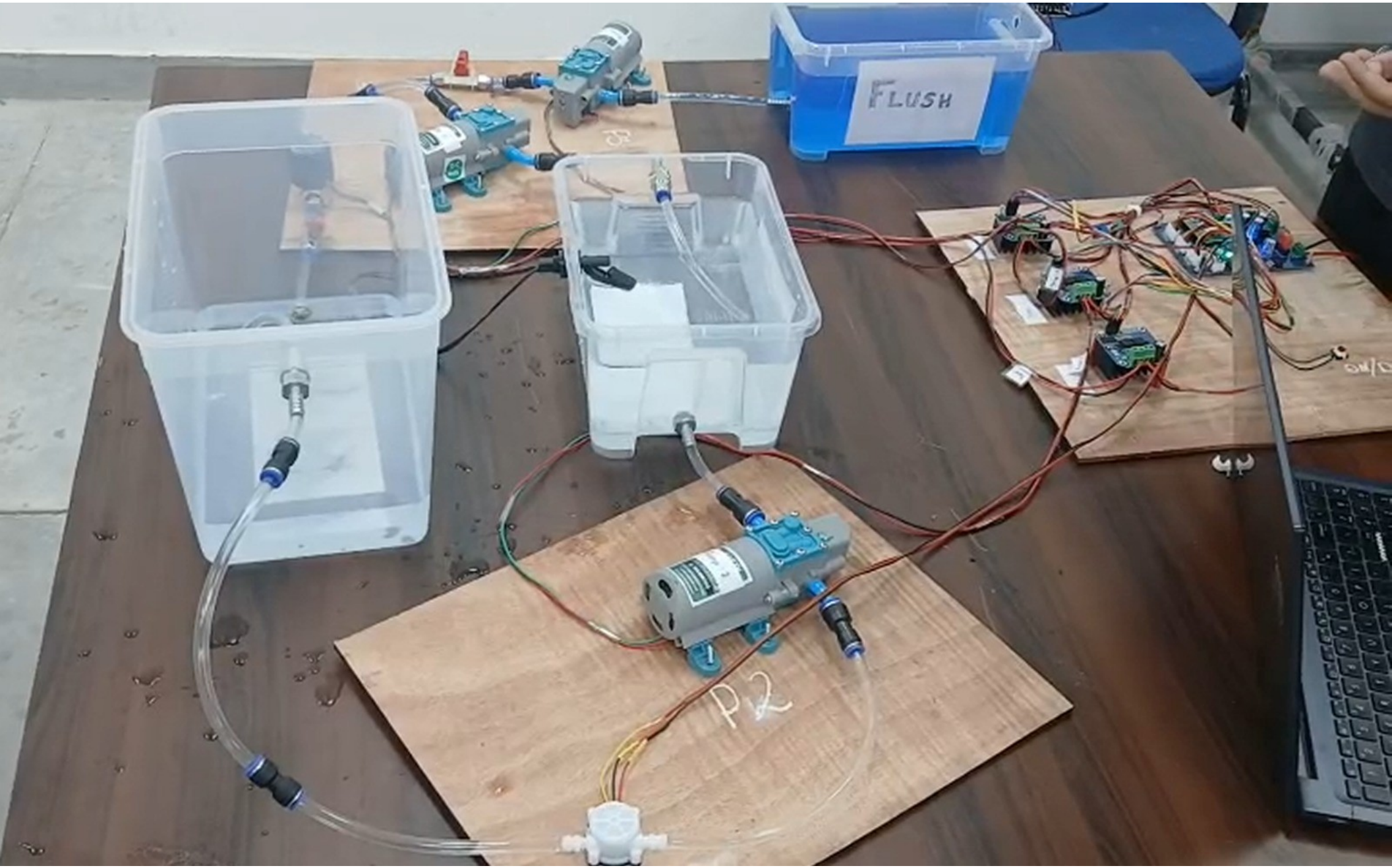
LRC-26-0217

Lam RESEARCH

P&ID SKETCH / WORKING SPACE



Team ID: LRC-26-0217 Team Name: PIKA


FLUSH
P2

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

⇒ Note: At max power, our pump delivered 2.6 L/min, so we have done our experiment for 1L/min, 2L/min & 2.6 L/min

⇒ At 1 L/min, $(A = \pi(5\times10^{-3})^2)$, $Q_{obs} = 0.989$ L/min.

$V_{th} = \frac{Q_{th}}{A_{pipe}} = \frac{10^{-3}/60}{\pi \times 25\times10^{-6}} = \frac{1000}{4712.38} = 0.212$ m/s

$V_{obs} = \frac{Q_{obs}}{A_{pipe}} = \frac{0.989/60}{78.5\times10^{-6}} = 0.209$ m/s

⇒ At 2 L/min, $Q_{obs} = 1.991$ L/min.

$V_{th} = \frac{Q_{th}}{A_{pipe}} = \frac{2\times10^{-3}/60}{78.5\times10^{-6}} = 0.424$ m/s

$V_{obs} = \frac{Q_{obs}}{A_{pipe}} = \frac{1.99\times10^{-3}/60}{78.5\times10^{-6}} = 0.422$ m/s

⇒ At 2.6 L/min, $Q_{obs} = 2.587$ L/min

$V_{th} = \frac{Q_{th}}{A_{pipe}} = \frac{2.6\times10^{-3}/60}{78.5\times10^{-6}} = 0.551$ m/s

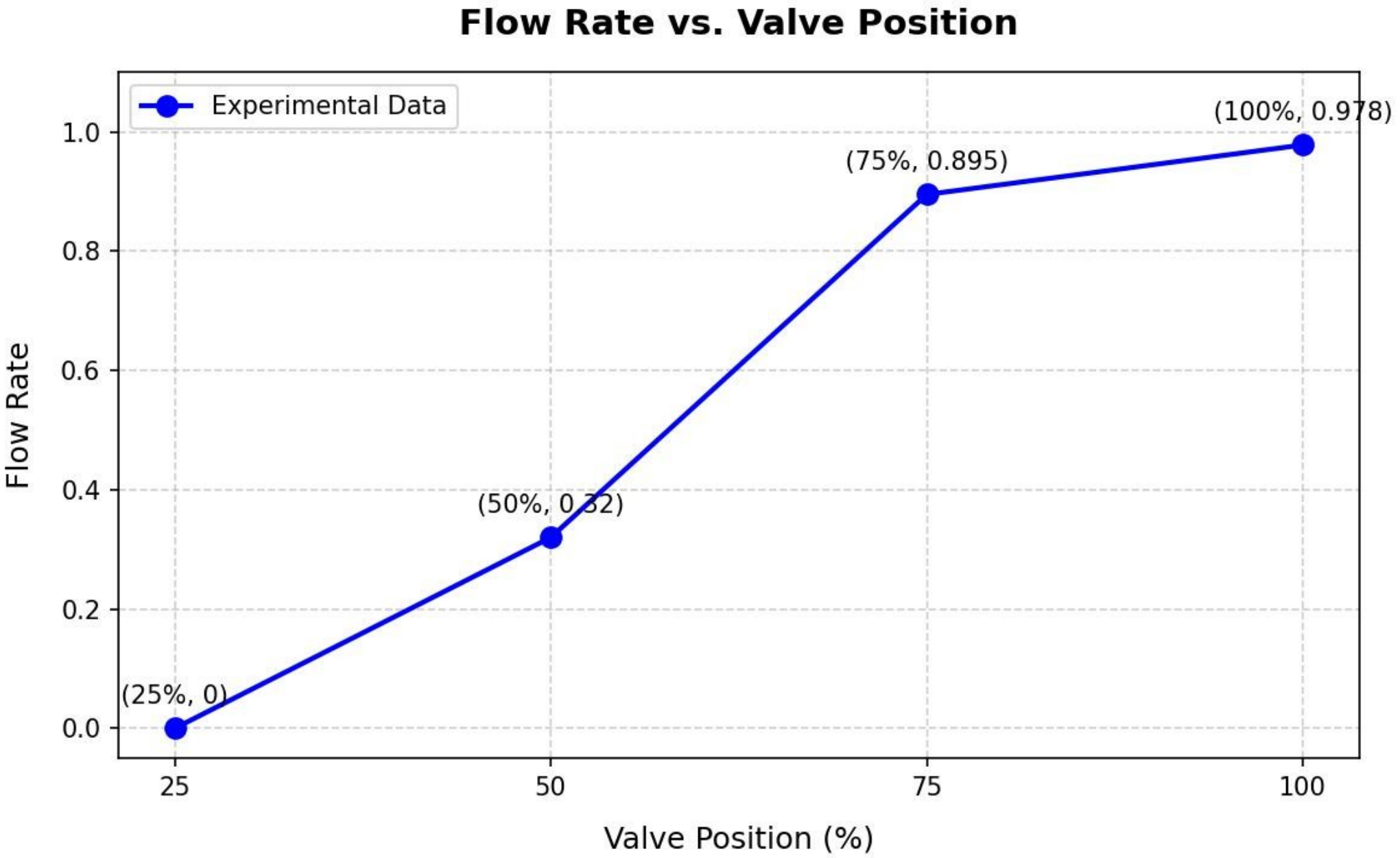
$V_{obs} = \frac{Q_{obs}}{A_{pipe}} = \frac{2.587\times10^{-3}/60}{78.5\times10^{-6}} = 0.548$ m/s

$E_1 = \frac{0.212-0.209}{0.212}\times100 = 1.41\%$

$E_2 = \frac{0.424-0.422}{0.424}\times100 = 0.471\%$

$E_3 = \frac{0.551-0.548}{0.551}\times100 = 0.54\%$

Avg Errors $= \frac{E_1+E_2+E_3}{3} = 0.807\%$


Flow Rate vs. Valve Position
Experimental Data
(100%, 0.978)
(75%, 0.895)
(50%, 0.32)
(25%, 0)
1.0
0.8
0.6
0.4
0.2
0.0
Flow Rate
25
50
75
100
Valve Position (%)

Lam RESEARCH

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0
50%	0.320
70%	0895
100%	0.978

GRAPH

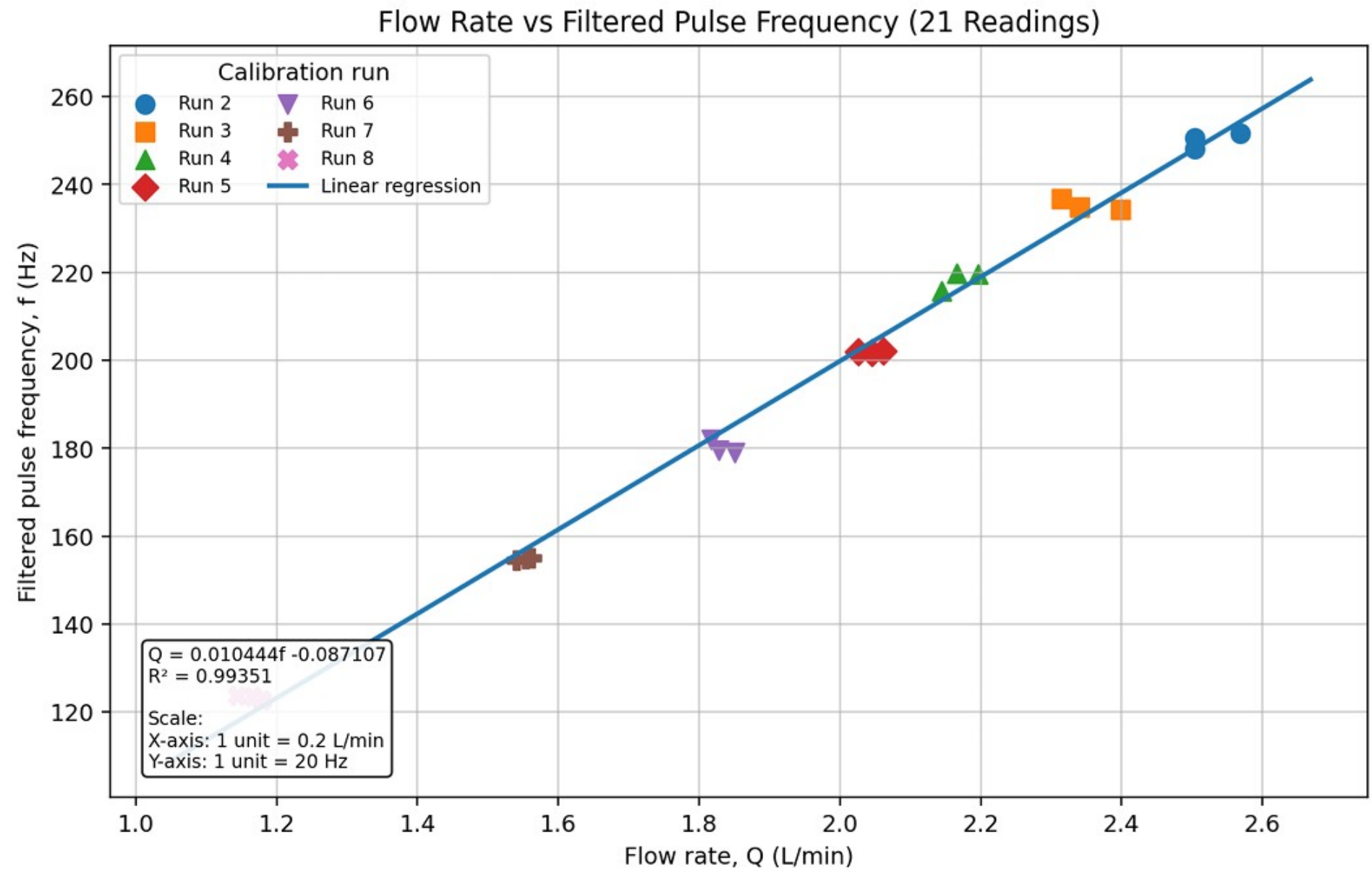
Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 =	0.993506
a =	0.0104642 $L.min^{-1}.Hz^{-1}$
b =	−0.0871075 $L.min^{-1}$
Maximum calibration error =	5.2325 %
Average calibration error =	1.6586 %

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	2939	11.68	251.602	0.500	2.5685
2	2932	12.51	234.350	0.500	2.3981
3	3019	13.99	215.7875	0.500	2.1444
4	2959	14.67	201.684	0.500	2.0450
5	2978	19.21	155.000	0.500	1.5961 7



Calibration equation: Q = 0.010444f -0.087107; R² = 0.99351.

CALCULATIONS / OBSERVATIONS

⇒ formula used :-

$$f = \frac{N}{t} \quad , \quad Q = \frac{V}{t} \times 60$$

where N = the number of pulses.
t = time in sec.
V = collected volume in liters.
f = frequency in Hz
Q = flow rate in L/min

⇒ Trial - 1 :-

V = 500 ml = 0.5 L
t = 11.68 sec
N of f = 2939

$f = \frac{N}{t} = \frac{2939}{11.68}$

f = 251.602 Hz

$Q = \frac{0.500}{11.68} \times 60$

Q = 2.5685 L/min

⇒ Trial - 2 :-

V = 500 ml = 0.5 L
t = 12.51 sec
N = 2932

$f = \frac{2932}{12.51}$

f = 234.350 Hz

$Q = \frac{0.5}{12.51} \times 60$

Q = 2.3981 L/min

⇒ Trial - 3 :-

V = 500 ml = 0.5 L
t = 13.99 sec
N = 3029

$f = \frac{3029}{13.99}$

f = 215.785 Hz

$Q = \frac{0.5}{13.99} \times 60$

Q = 2.1444 L/min

⇒ same for trial - 4 & 5.

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

Given: $Q = 0.5L$, $V = 2.5\,L/min$

$\therefore T_H = \frac{Q}{V} = \frac{0.5}{2.5}\,min = 0.2\,min = 12\,sec.$

$\therefore T_H = 12sec$

1) Residence time is the characteristic time required for fluid to pass through the given tubing volume at the specified flow rate. In this system, it shows the transport delay between a change occuring in the upstream and its corresponding reaction captured by the flow sensor FT-01 downstream. Therefore, it is an important parameter for analyzing the dynamic response of the hydraulic system.

2) Residence time is directly proportional to tubing volume at a constant flow rate. Thus, when Q is doubled, $T_H = \frac{1}{2.5}\,min = 0.4\,min = 24sec$. Therefore the residence time is doubled and it takes more time for any change in upstream to be captured by FT-01 in the downstream.

3) Residence time is inversely proportional to volume flow rate at a constant tubing volume. Thus, when V is doubled, $T_H = \frac{0.5}{5}\,min = 0.1\,min = 6sec$. Hence doubling the volume flow rate, halves the residence time.

4) Residence time introduces a transport delay between upstream disturbance and its observation at FT-01. Therefore a sudden change in valve condition may not immediately appear in the measured flow signal. Knowledge of residence time helps distinguish an actual hydraulic ~~system~~ fault from a normal delayed response of the fluid system and prevents premature fault detection.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

CONDITION A:
The pump is on and the valve is open, producing a normal FT-01 pulse signal and normal measured flow. This indicates that the hydraulic path and flow measurement system are operating under the expected condition.

CONDITION B:
Partial closure of valve increases hydraulic resistance of the flow path. Consequently, the volumetric flow rate decreases producing a corresponding reduction in FT-01 pulse frequency. Thus, with reduced flow, consistency is maintained w/ the specified valve position

CONDITION C:
The pump is on and valve is OPEN, but FT-01 produces no pulses and the measured flow is zero.
Physical interpretations: → Pump failure (shown on on interface but not pumping fluid, due to air lock)
→ Blockage / leakage in the upstream restricts flow showing zero pulses
→ FT-01 attached in the opposite direction thus not recording pulses or being incorrectly connected.

CONDITION D:
The pump is OFF while valve is open. Since pump is not providing the pressure head required to drive circulation, zero flow and zero pulses are expected & consequently recorded. Not a hydraulic fault.

CONDITION E:
The pump is ON and the valve is OPEN, but pulses in FT01 are intermittent. This indicates:
→ air entrainment, cavitation in pump, intermittent blocking, pump instability, faulty sensor.

The most difficult to distinguish using FT-01 alone is the condition C. It does not provide a strong case on physical cause and can be a result from pump failure, hydraulic blockage, inadequate supply, high head, or failure of FT-01 sensor itself. Therefore, FT-01 should be used as the primary flow-deviation indicator, while pump status, fluid level and other parameters must be combined for reliable fault isolation.

```
// ======================================================================
// Lam Research Challenge 3.0 - Stage 2 - Task 1 (Case 1 + Case 2)
// Automatic recirculation, sump level protection, flow totalizing, flush
// sequence and auto shutdown.
//
// SEQUENCE (fully automatic after the ON/OFF push button is pressed):
//
//  CASE 1  - Solenoid OV-01 OPEN, Pump-01 @50%, Pump-02 @100%
//          Float switch (sump HIGH) -> Pump-01 OFF, Pump-02 keeps running
//          Float clears (level reduced) -> Pump-01 back ON @50%
//          FT-01 flow is converted with Q = a*f + b, logged in mL/s,
//          L/min and totalised in litres.
//  CASE 2  - Starts automatically after CASE1_RUN_SEC:
//          Solenoid OV-01 CLOSED (isolates the 10 L tank),
//          Pump-03 ON (flush tank -> equal tee), Pump-01 stays OFF
//          (no suction source while OV-01 is closed), Pump-02 stays on.
//          When the flush tank runs dry Pump-03 loses its load and its
//          motor current (driver R_IS) falls suddenly; once the
//          drop is confirmed everything switches OFF (state DONE).
//  ON/OFF is a MOMENTARY PUSH BUTTON (toggle on each press):
//    IDLE  + press -> start Case 1
//    RUNNING + press -> everything stops immediately, back to IDLE
//    DONE  + press -> back to IDLE (press again to start a new cycle)
// ======================================================================

#include <WiFi.h>
#include <WebServer.h>

// ---------------- WiFi ----------------
const char* ssid     = "TinkerersLab_IITRopar";
const char* password = "TinkerersLab@2024";
const unsigned long WIFI_TIMEOUT_MS = 15000;   // control works even w/o WiFi

WebServer server(80);

// ==================== PIN MAP ====================
const int FLOW_SENSOR_PIN  = 14;   // YF-S401 signal

// Float switch (GPIO35 input-only, needs EXTERNAL pull resistor)
const int LEVEL_SWITCH_PIN = 35;
const int LEVEL_SWITCH_ACTIVE_STATE = LOW;     // LOW = sump level HIGH

// ON/OFF momentary push button.
const int ONOFF_SWITCH_PIN = 34;
const int ONOFF_SWITCH_ACTIVE_STATE = LOW;     // LOW = button pressed
const unsigned long BTN_DEBOUNCE_MS = 50;

// Solenoid / ON-OFF valve OV-01 (MOSFET module)
const int SOLENOID_PIN         = 27;
```

```
const int SOLENOID_PWM_FREQ    = 20000;
const int SOLENOID_HIT_DUTY    = 255;
const int SOLENOID_HOLD_DUTY   = 90;
const int SOLENOID_HIT_TIME_MS = 80;
const int SOLENOID_PWM_CH      = 4;

// Pump 1 (Driver 1)
const int P3_RPWM = 33, P3_LPWM = 13, P3_REN = 26, P3_LEN = 26; //We assigned one of the
enable pins to R_IS to detect current drawn by pump
// Pump 2 (Driver 2)
const int P2_RPWM = 18, P2_LPWM = 19, P2_REN = 17, P2_LEN = 16;
// Pump 3 (Driver 3)
const int P1_RPWM = 25, P1_LPWM = 23, P1_REN = 22, P1_LEN = 21;

const int P3_R_IS = 32;   // VP

const int PWM_FREQ = 5000;
const int PWM_RES  = 8;
const int P1_PWM_CH = 0;
const int P2_PWM_CH = 1;
const int P3_PWM_CH = 2;

// ==================== PROCESS SETTINGS ====================
const int PUMP1_DUTY       = 128;   // 50 % duty (Case 1 requirement)
const int PUMP2_DUTY       = 255;   // 100 % duty
const int PUMP3_FLUSH_DUTY = 255;   // flush pump

// Calibration  Q[L/min] = CAL_A * f[Hz] + CAL_B

const float CAL_A = 0.01044420f;
const float CAL_B = -0.08710746f;

// Float switch conditioning
const unsigned long FLOAT_DEBOUNCE_MS   = 200;   // must be stable this long
const unsigned long FLOAT_CLEAR_HOLD_MS = 400;  // keep Pump-1 off a bit
                                        // after level has dropped

// Case 1 -> Case 2 automatic hand-over
const unsigned long CASE1_RUN_SEC = 180;         // run Case 1 this long

// Window between Case 1 and Case 2 for the operator to manually open the
// ball valve on the flush line (OV-01 is closed, Pump-01/03 are off, Pump-02
// keeps draining the sump). Pump-03 starts automatically when it expires.
const unsigned long VALVE_WAIT_SEC = 10;

// Flush end detection (flush tank empty -> flow collapses)
const unsigned long FLUSH_GRACE_MS   = 10000;    // ignore start-up inrush / priming
const unsigned long FLUSH_MAX_MS     = 300000UL; // safety timeout (5 min)
```

```
// Pump-3 current-drop detection (flush tank empty)
const unsigned long CS_SAMPLE_MS      = 20;      // current sampling period
const int           CS_OVERSAMPLE     = 8;       // ADC reads averaged per sample
const float         CS_FAST_ALPHA     = 0.25f;   // fast filter (~80 ms)
const float         CS_SLOW_ALPHA     = 0.02f;   // baseline filter (~1 s)
const float         CS_DROP_FRACTION  = 0.30f;   // fall > 30 % below baseline ...
const float         CS_DROP_MIN_MV    = 40.0f;   // ... and > 40 mV (noise guard)
const float         CS_MIN_BASELINE_MV = 100.0f; // baseline too small = sensor not valid
const int           CS_DROP_CONFIRM   = 10;      // consecutive samples (10 x 20 ms)

// Sampling / logging
const unsigned long SAMPLE_INTERVAL_MS = 1000;
const int MAX_SAMPLES = 3000;

// ==================== STATE MACHINE ====================
enum SysState : uint8_t { ST_IDLE = 0, ST_CASE1, ST_VALVE_WAIT, ST_FLUSH, ST_DONE };
SysState sysState = ST_IDLE;

const char* stateName(SysState s) {
  switch (s) {
    case ST_IDLE:  return "IDLE";
    case ST_CASE1: return "CASE 1: RECIRCULATION";
    case ST_VALVE_WAIT: return "VALVE WAIT: OPEN BALL VALVE";
    case ST_FLUSH: return "CASE 2: FLUSHING";
    case ST_DONE:  return "DONE - SYSTEM OFF";
  }
  return "?";
}

// ==================== DATA ====================
volatile unsigned long pulseCount = 0;

struct DataPoint {
  uint32_t timeMs;
  float    lpm;
  float    mlps;
  float    totalL;
  float    isMv;      // Pump-3 current-sense voltage
};

DataPoint logData[MAX_SAMPLES];
int dataCount = 0;

unsigned long runStartTime   = 0;   // switch turned ON
unsigned long flushStartTime = 0;
unsigned long valveWaitStart = 0;
unsigned long lastSampleTime = 0;
unsigned long lastSamplePulses = 0;
```

```
float currentLpm   = 0.0f;
float currentMlps  = 0.0f;
float totalLiters  = 0.0f;

// Pump-3 current sense
float pump3IsMv   = 0.0f;   // raw R_IS reading in mV
float isFastMv    = 0.0f;   // fast filtered
float isSlowMv    = 0.0f;   // slow baseline
int   dropCount   = 0;
bool  currentDropDetected = false;
unsigned long lastCsTime = 0;

bool solenoidOpen = false;
int  pump1Duty = 0, pump2Duty = 0, pump3Duty = 0;

// Conditioned inputs
bool floatHigh      = false;   // debounced: sump level HIGH
bool floatRawLast   = false;
unsigned long floatChangeTime = 0;
unsigned long floatClearTime  = 0;
bool pump1Held      = false;   // Pump-1 stopped by level protection

bool btnPressed     = false;   // debounced ON/OFF push-button level
bool btnRawLast     = false;
bool btnEvent       = false;   // set once per new press, consumed by state machine
unsigned long btnChangeTime = 0;

// ==================== ISR ====================
void IRAM_ATTR pulseISR() { pulseCount++; }

unsigned long getPulseCount() {
  noInterrupts();
  unsigned long c = pulseCount;
  interrupts();
  return c;
}

// ==================== ACTUATORS ====================
void setPump(int pump, int duty) {
  duty = constrain(duty, 0, 255);
  if (pump == 1 && duty != pump1Duty) { pump1Duty = duty; ledcWrite(P1_PWM_CH, duty); }
  if (pump == 2 && duty != pump2Duty) { pump2Duty = duty; ledcWrite(P2_PWM_CH, duty); }
  if (pump == 3 && duty != pump3Duty) { pump3Duty = duty; ledcWrite(P3_PWM_CH, duty); }
}

void openSolenoid() {
  if (solenoidOpen) return;
  ledcWrite(SOLENOID_PWM_CH, SOLENOID_HIT_DUTY);
  delay(SOLENOID_HIT_TIME_MS);
```

```
  ledcWrite(SOLENOID_PWM_CH, SOLENOID_HOLD_DUTY);
  solenoidOpen = true;
  Serial.println("Solenoid OPEN");
}

void closeSolenoid() {
  ledcWrite(SOLENOID_PWM_CH, 0);
  if (solenoidOpen) Serial.println("Solenoid CLOSED");
  solenoidOpen = false;
}

void allOff() {
  setPump(1, 0);
  setPump(2, 0);
  setPump(3, 0);
  closeSolenoid();
}

// ==================== INPUT CONDITIONING ====================
void updateInputs() {
  unsigned long now = millis();

  // ---- float switch (debounce) ----
  bool rawFloat = (digitalRead(LEVEL_SWITCH_PIN) == LEVEL_SWITCH_ACTIVE_STATE);
  if (rawFloat != floatRawLast) {
    floatRawLast = rawFloat;
    floatChangeTime = now;
  }
  if ((now - floatChangeTime) >= FLOAT_DEBOUNCE_MS && rawFloat != floatHigh) {
    floatHigh = rawFloat;
    if (!floatHigh) floatClearTime = now;   // level just dropped
  }

  // ---- ON/OFF push button (debounce + press-edge detect) ----
  bool rawBtn = (digitalRead(ONOFF_SWITCH_PIN) == ONOFF_SWITCH_ACTIVE_STATE);
  if (rawBtn != btnRawLast) {
    btnRawLast = rawBtn;
    btnChangeTime = now;
  }
  if ((now - btnChangeTime) >= BTN_DEBOUNCE_MS && rawBtn != btnPressed) {
    btnPressed = rawBtn;
    if (btnPressed) btnEvent = true;     // one event per press (no auto-repeat)
  }
}

// ---- Pump-3 current sense ----------------------------------------
// BTS7960 IS output is proportional to load current. Driving RPWM puts
// the current through the R half-bridge, so only R_IS is monitored.
float readIsMv(int pin) {
```

```
  uint32_t sum = 0;
  for (int i = 0; i < CS_OVERSAMPLE; i++) sum += analogReadMilliVolts(pin);
  return sum / (float)CS_OVERSAMPLE;
}

void resetCurrentSense() {
  float v = readIsMv(P3_R_IS);
  pump3IsMv = isFastMv = isSlowMv = v;
  dropCount = 0;
  currentDropDetected = false;
  lastCsTime = millis();
}

// Sudden-fall detector: fast signal drops well below the slow baseline
// and stays there for CS_DROP_CONFIRM consecutive samples.
void updateCurrentSense() {
  unsigned long now = millis();
  if (now - lastCsTime < CS_SAMPLE_MS) return;
  lastCsTime = now;

  pump3IsMv = readIsMv(P3_R_IS);
  isFastMv += CS_FAST_ALPHA * (pump3IsMv - isFastMv);

  bool armed = (sysState == ST_FLUSH) &&
               (now - flushStartTime > FLUSH_GRACE_MS) &&
               (isSlowMv >= CS_MIN_BASELINE_MV);

  if (armed) {
    float drop = isSlowMv - isFastMv;
    if (drop > CS_DROP_MIN_MV && drop > CS_DROP_FRACTION * isSlowMv) dropCount++;
    else dropCount = 0;
    if (dropCount >= CS_DROP_CONFIRM) currentDropDetected = true;
  } else {
    dropCount = 0;
  }

  // Baseline follows the load only while no drop is developing,
  // so a falling current is never absorbed into the reference.
  if (dropCount == 0) isSlowMv += CS_SLOW_ALPHA * (isFastMv - isSlowMv);
}

// Level protection for Pump-1: stop while sump is high, resume after the
// level has been low for FLOAT_CLEAR_HOLD_MS.
bool pump1Allowed() {
  if (floatHigh) {
    pump1Held = true;
    return false;
  }
  if (pump1Held && (millis() - floatClearTime) < FLOAT_CLEAR_HOLD_MS) {
```

```
    return false;
  }
  pump1Held = false;
  return true;
}

// ==================== SEQUENCE CONTROL ====================
void startRun() {
  dataCount = 0;
  totalLiters = 0.0f;
  currentLpm = currentMlps = 0.0f;

  runStartTime = millis();
  lastSampleTime = runStartTime;
  lastSamplePulses = getPulseCount();

  sysState = ST_CASE1;
  openSolenoid();                // 10 L tank -> Pump-01
  setPump(2, PUMP2_DUTY);        // Pump-02 always 100 %
  setPump(1, pump1Allowed() ? PUMP1_DUTY : 0);

  Serial.println("\n===== CASE 1 STARTED =====");
  Serial.println("Time(s),Flow(L/min),Flow(mL/s),Total(L),P3_IS(mV),State");
}

void startValveWait() {
  sysState = ST_VALVE_WAIT;
  valveWaitStart = millis();
  closeSolenoid();       // isolate the 10 L process tank
  setPump(1, 0);         // no suction source now -> do not run dry
  setPump(2, PUMP2_DUTY); // keep draining the sump through FT-01
  Serial.println("\n>>> OPEN THE FLUSH-LINE BALL VALVE NOW <<<");
}

void startFlush() {
  sysState = ST_FLUSH;
  flushStartTime = millis();

  closeSolenoid();               // isolate the 10 L process tank
  setPump(3, PUMP3_FLUSH_DUTY);   // flush tank -> equal tee
  resetCurrentSense();           // seed current baseline for Pump-3
  // Pump-01 stays OFF (OV-01 is closed, it would run dry); Pump-02 keeps running
  Serial.println("\n===== CASE 2 (FLUSH) STARTED =====");
}

void finishRun(const char* reason) {
  allOff();
  sysState = ST_DONE;
  Serial.print("===== SYSTEM OFF: ");
```

```
  Serial.print(reason);
  Serial.print(" | Total transferred = ");
  Serial.print(totalLiters, 3);
  Serial.println(" L =====");
}

void abortToIdle() {
  allOff();
  sysState = ST_IDLE;
  pump1Held = false;
  Serial.println("ON/OFF button pressed -> STOPPED, IDLE");
}

// One-second flow sample: pulses -> frequency -> L/min -> mL/s -> total
void takeSample(unsigned long now) {
  unsigned long pulses = getPulseCount();
  unsigned long dPulses = pulses - lastSamplePulses;
  float dt = (now - lastSampleTime) / 1000.0f;
  lastSamplePulses = pulses;
  lastSampleTime = now;
  if (dt <= 0.0f) return;

  float freq = dPulses / dt;                        // f = N / t
  float lpm  = (dPulses == 0) ? 0.0f : (CAL_A * freq + CAL_B);
  if (lpm < 0.0f) lpm = 0.0f;

  currentLpm  = lpm;
  currentMlps = lpm * 1000.0f / 60.0f;
  totalLiters += lpm * dt / 60.0f;                  // integrate Q dt

  if (dataCount < MAX_SAMPLES) {
    logData[dataCount].timeMs = now - runStartTime;
    logData[dataCount].lpm    = currentLpm;
    logData[dataCount].mlps   = currentMlps;
    logData[dataCount].totalL = totalLiters;
    logData[dataCount].isMv   = pump3IsMv;
    dataCount++;
  }

  Serial.printf("%.1f,%.3f,%.2f,%.3f,%.0f,%s\n",
                (now - runStartTime) / 1000.0f,
                currentLpm, currentMlps, totalLiters, pump3IsMv,
                stateName(sysState));
}

void runStateMachine() {
  unsigned long now = millis();

  // ON/OFF push button has top priority (toggle behaviour)
```

```
if (btnEvent) {
  btnEvent = false;
  if (sysState == ST_IDLE) {
    startRun();                    // press -> start Case 1
    return;
  } else if (sysState == ST_DONE) {
    sysState = ST_IDLE;            // acknowledge; outputs are already off
    Serial.println("Button pressed -> IDLE. Press again to start.");
    return;
  } else {
    abortToIdle();                 // press while running -> stop
    return;
  }
}

switch (sysState) {

  case ST_IDLE:
    break;                         // wait for a button press

  case ST_CASE1:
    // Pump-02 always on; Pump-01 follows the float switch
    setPump(2, PUMP2_DUTY);
    setPump(1, pump1Allowed() ? PUMP1_DUTY : 0);

    if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) takeSample(now);

    if (now - runStartTime >= CASE1_RUN_SEC * 1000UL) startValveWait();
    break;

  case ST_VALVE_WAIT:
    setPump(1, 0);
    setPump(2, PUMP2_DUTY);
    if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) takeSample(now);
    if (now - valveWaitStart >= VALVE_WAIT_SEC * 1000UL) startFlush();
    break;

  case ST_FLUSH:
    setPump(2, PUMP2_DUTY);
    setPump(3, PUMP3_FLUSH_DUTY);
    setPump(1, 0);                 // OV-01 closed: Pump-01 must stay OFF

    if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) takeSample(now);

    // Flush tank empty = sudden fall in Pump-3 motor current
    if (currentDropDetected) {
      finishRun("flush tank empty (Pump-3 current drop)");
      return;
    }
```

```
      if (now - flushStartTime > FLUSH_MAX_MS) finishRun("flush timeout");
      break;

    case ST_DONE:
      // stay OFF until the button is pressed
      break;
  }
}

// ==================== WEB DASHBOARD ====================
const char index_html[] PROGMEM = R"HTML(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Lam Challenge - Task 1</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<style>
  body{font-family:Arial,sans-serif;text-align:center;margin:20px
auto;max-width:720px;background:#121212;color:#fff}
  .card{background:#1e1e1e;padding:18px;border-radius:12px;margin-bottom:18px;box-shadow:0 4px
10px rgba(0,0,0,.5)}
  .big{font-size:30px;font-weight:bold;color:#00bcd4;margin:6px 0}
  .val{font-weight:bold;color:#00e676}
  .row{display:flex;justify-content:space-around;flex-wrap:wrap}
  .row div{margin:6px 10px}
  button{border:none;border-radius:8px;padding:12px
22px;font-size:16px;cursor:pointer;background:#1565c0;color:#fff}

.dot{display:inline-block;width:16px;height:16px;border-radius:50%;background:#777;vertical-align:middle;
margin-right:6px}
  .on{background:#00e676;box-shadow:0 0 8px #00e676}
  .off{background:#d50000;box-shadow:0 0 8px #d50000}
</style>
</head>
<body>
<h2>Task 1 &ndash; Recirculation + Flush</h2>

<div id="waitBanner" class="card" style="display:none;background:#ff6f00;color:#000">
  <h2 style="margin:4px">&#9888; OPEN THE FLUSH BALL VALVE NOW</h2>
  <div class="big" style="color:#000"><span id="waitLeft">0</span> s until Pump-03 starts</div>
</div>

<div class="card">
  <h3>State: <span id="state" class="val">--</span></h3>
  <div class="row">
    <div>Flow<div class="big"><span id="lpm">0.00</span> L/min</div></div>
    <div>Flow<div class="big"><span id="mlps">0.0</span> mL/s</div></div>
    <div>Total to 10 L tank<div class="big"><span id="total">0.000</span> L</div></div>
```

```
  </div>
  <p>Elapsed: <span id="elapsed">0</span> s</p>
  <button onclick="location.href='/download'">Download CSV</button>
</div>

<div class="card">
  <canvas id="mlpsChart" style="max-height:260px"></canvas>
</div>
<div class="card">
  <canvas id="lpmChart" style="max-height:260px"></canvas>
</div>

<div class="card">
  <div class="row">
    <div><span id="swDot" class="dot"></span>System running</div>
    <div><span id="flDot" class="dot"></span>Sump HIGH (float)</div>
    <div><span id="ovDot" class="dot"></span>OV-01 valve</div>
  </div>
  <div class="row">
    <div>Pump-1: <span id="p1" class="val">0</span></div>
    <div>Pump-2: <span id="p2" class="val">0</span></div>
    <div>Pump-3: <span id="p3" class="val">0</span></div>
    <div>P3 current sense: <span id="ism" class="val">0</span> mV</div>
  </div>
</div>

<script>
function mk(id,label,color,ytitle){
  return new Chart(document.getElementById(id).getContext('2d'),{
    type:'line',
    data:{labels:[],datasets:[{label:label,data:[],borderColor:color,borderWidth:2,fill:false,tension:.2}]},
    options:{animation:false,
      scales:{x:{title:{display:true,text:'Time (s)',color:'#888'},ticks:{color:'#888'}},
              y:{beginAtZero:true,title:{display:true,text:ytitle,color:'#888'},ticks:{color:'#888'}}},
      plugins:{legend:{labels:{color:'#fff'}}}}});
}
const c1 = mk('mlpsChart','Flow (mL/s)','#00bcd4','Flow (mL/s)');
const c2 = mk('lpmChart','Flow (L/min)','#ffb300','Flow (L/min)');
let lastIdx = -1;

function dot(id,on,bad){document.getElementById(id).className='dot '+(on?(bad?'off':'on'):'');}

function upd(){
  fetch('/data').then(r=>r.json()).then(d=>{
    state.innerText=d.state; lpm.innerText=d.lpm.toFixed(2);
    mlps.innerText=d.mlps.toFixed(1); total.innerText=d.total.toFixed(3);
    elapsed.innerText=d.elapsed.toFixed(0);
    waitBanner.style.display = d.wait_left>0 ? 'block' : 'none';
    waitLeft.innerText = d.wait_left.toFixed(0);
```

```
    p1.innerText=d.p1; p2.innerText=d.p2; p3.innerText=d.p3; ism.innerText=d.is_mv.toFixed(0);
    dot('swDot',d.sw,false); dot('flDot',d.level_high,true); dot('ovDot',d.valve,false);

    if(d.sample_index<lastIdx){[c1,c2].forEach(c=>{c.data.labels=[];c.data.datasets[0].data=[];});}
    if(d.sample_index>=0 && d.sample_index!==lastIdx){
      const t=d.sample_time.toFixed(1);
      c1.data.labels.push(t); c1.data.datasets[0].data.push(d.mlps_s);
      c2.data.labels.push(t); c2.data.datasets[0].data.push(d.lpm_s);
      c1.update(); c2.update();
    }
    lastIdx=d.sample_index;
  }).catch(e=>console.log(e));
}
setInterval(upd,1000); upd();
</script>
</body>
</html>
)HTML";

// ==================== WEB HANDLERS ====================
void handleRoot() { server.send(200, "text/html", index_html); }

void handleGetData() {
  int idx = dataCount - 1;
  float st = 0, lpmS = 0, mlpsS = 0;
  if (idx >= 0) {
    st    = logData[idx].timeMs / 1000.0f;
    lpmS  = logData[idx].lpm;
    mlpsS = logData[idx].mlps;
  }
  float elapsed = (sysState == ST_IDLE) ? 0.0f
                                        : (millis() - runStartTime) / 1000.0f;
  if (sysState == ST_DONE && idx >= 0) elapsed = st;

  String j = "{";
  j += "\"state\":\"" + String(stateName(sysState)) + "\"";
  j += ",\"lpm\":" + String(currentLpm, 3);
  j += ",\"mlps\":" + String(currentMlps, 2);
  j += ",\"total\":" + String(totalLiters, 4);
  j += ",\"elapsed\":" + String(elapsed, 1);
  float waitLeft = 0.0f;
  if (sysState == ST_VALVE_WAIT) {
    long left = (long)(VALVE_WAIT_SEC * 1000UL) - (long)(millis() - valveWaitStart);
    waitLeft = (left > 0 ? left : 0) / 1000.0f;
  }
  j += ",\"wait_left\":" + String(waitLeft, 1);
  j += ",\"p1\":" + String(pump1Duty);
  j += ",\"p2\":" + String(pump2Duty);
  j += ",\"p3\":" + String(pump3Duty);
```

```
  j += ",\"is_mv\":" + String(pump3IsMv, 0);
  j += ",\"valve\":" + String(solenoidOpen ? "true" : "false");
  bool running = (sysState == ST_CASE1 || sysState == ST_VALVE_WAIT || sysState == ST_FLUSH);
  j += ",\"sw\":" + String(running ? "true" : "false");
  j += ",\"level_high\":" + String(floatHigh ? "true" : "false");
  j += ",\"sample_index\":" + String(idx);
  j += ",\"sample_time\":" + String(st, 2);
  j += ",\"lpm_s\":" + String(lpmS, 3);
  j += ",\"mlps_s\":" + String(mlpsS, 2);
  j += "}";
  server.send(200, "application/json", j);
}

void handleDownloadCSV() {
  if (dataCount == 0) {
    server.send(404, "text/plain", "No data available.");
    return;
  }
  String csv = "Time_s,Flow_L_per_min,Flow_mL_per_s,Total_L,P3_IS_mV\n";
  for (int i = 0; i < dataCount; i++) {
    csv += String(logData[i].timeMs / 1000.0f, 2) + ",";
    csv += String(logData[i].lpm, 3) + ",";
    csv += String(logData[i].mlps, 2) + ",";
    csv += String(logData[i].totalL, 4) + ",";
    csv += String(logData[i].isMv, 0) + "\n";
  }
  server.sendHeader("Content-Disposition",
                    "attachment; filename=task1_flow_data.csv");
  server.send(200, "text/csv", csv);
}

// ==================== SETUP ====================
void setup() {
  Serial.begin(115200);

  // Flow sensor
  pinMode(FLOW_SENSOR_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), pulseISR, RISING);

  // Float switch + ON/OFF switch (input-only pins, external pulls)
  pinMode(LEVEL_SWITCH_PIN, INPUT);
  pinMode(ONOFF_SWITCH_PIN, INPUT);

  // Pump-3 current-sense input (ADC1, full 0-3.3 V range)
  pinMode(P3_R_IS, INPUT);
  analogSetPinAttenuation(P3_R_IS, ADC_11db);

  // IBT-2 driver pins
  int digitalOutputs[] = {
```

```
  P1_LPWM, P1_REN, P1_LEN,
  P2_LPWM, P2_REN, P2_LEN,
  P3_LPWM, P3_REN, P3_LEN
};
for (int i = 0; i < 9; i++) pinMode(digitalOutputs[i], OUTPUT);

Serial2.end();   // release GPIO16/17

digitalWrite(P1_LPWM, LOW);
digitalWrite(P2_LPWM, LOW);
digitalWrite(P3_LPWM, LOW);

digitalWrite(P1_REN, HIGH); digitalWrite(P1_LEN, HIGH);
digitalWrite(P2_REN, HIGH); digitalWrite(P2_LEN, HIGH);
digitalWrite(P3_REN, HIGH); digitalWrite(P3_LEN, HIGH);

// Pump PWM
ledcSetup(P1_PWM_CH, PWM_FREQ, PWM_RES);
ledcSetup(P2_PWM_CH, PWM_FREQ, PWM_RES);
ledcSetup(P3_PWM_CH, PWM_FREQ, PWM_RES);
ledcAttachPin(P1_RPWM, P1_PWM_CH);
ledcAttachPin(P2_RPWM, P2_PWM_CH);
ledcAttachPin(P3_RPWM, P3_PWM_CH);
ledcWrite(P1_PWM_CH, 0);
ledcWrite(P2_PWM_CH, 0);
ledcWrite(P3_PWM_CH, 0);

// Solenoid PWM
ledcSetup(SOLENOID_PWM_CH, SOLENOID_PWM_FREQ, PWM_RES);
ledcAttachPin(SOLENOID_PIN, SOLENOID_PWM_CH);
closeSolenoid();

// Initialise debounced inputs
floatRawLast  = floatHigh =
   (digitalRead(LEVEL_SWITCH_PIN) == LEVEL_SWITCH_ACTIVE_STATE);
btnRawLast = btnPressed =
   (digitalRead(ONOFF_SWITCH_PIN) == ONOFF_SWITCH_ACTIVE_STATE);
btnEvent = false;                  // a button held at boot does not start a run

// WiFi (optional - control logic does not depend on it)
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
unsigned long t0 = millis();
while (WiFi.status() != WL_CONNECTED && millis() - t0 < WIFI_TIMEOUT_MS) {
  delay(500);
  Serial.print(".");
}
if (WiFi.status() == WL_CONNECTED) {
  Serial.println("\nWiFi Connected!");
```

```
    Serial.print("Dashboard URL: http://");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\nWiFi not connected - running without dashboard.");
  }

  server.on("/", handleRoot);
  server.on("/data", handleGetData);
  server.on("/download", handleDownloadCSV);
  server.begin();

  Serial.println("Ready. Press the ON/OFF button to start Case 1.");
}

// ==================== LOOP ====================
void loop() {
  // keep driver enables high
  digitalWrite(P1_REN, HIGH); digitalWrite(P1_LEN, HIGH);
  digitalWrite(P2_REN, HIGH); digitalWrite(P2_LEN, HIGH);
  digitalWrite(P3_REN, HIGH); digitalWrite(P3_LEN, HIGH);

  server.handleClient();
  updateInputs();
  updateCurrentSense();
  runStateMachine();
}
```